Conclusion

Summary I

This paper has presented `template_pools_rules_tools`, a meta-project exemplar demonstrating how research software projects embedded in a monorepo can integrate three categories of shared resources — data pools (fonds), governance rules, and executable tools — without tight coupling to any specific resource instance. The key contributions are:

Summary I

1. **A four-module architecture** (`fonds_reader`, `rules_applier`, `tools_invoker`, `integration`) that provides a canonical pattern for resource-aware project code in the template repository. Each module is independently testable and independently deployable.
2. **A typed manifest convention** (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that makes resource capabilities explicit and checkable at pipeline initialisation time, shifting failure detection from runtime to startup — a significant improvement for reproducibility (Wilson et al. 2014).
3. **A token injection pipeline** that links manuscript prose to integration runtime statistics through doubled-brace, uppercase-key placeholders, ensuring that quantitative claims in the manuscript are always generated by the pipeline rather than authored manually. In the current run this covered 3 fonds, 2 rule sets, 4 tools, and 8 bibliography entries.
4. **A three-level resilience design** — resource absence, schema malformation, and script absence — that allows the pipeline to degrade gracefully and report failures informatively rather than crashing, consistent with best practices for robust research software (Taschuk and Wilson 2017).

Design Decisions Revisited I

Several design decisions deserve emphasis as lessons for future exemplar authors:

Repo-root-relative discovery is non-negotiable in a forkable monorepo. Any hard-coded absolute path or working-directory assumption will fail when the repository is cloned to a different location. The `pathlib.Path(__file__).resolve().parents[N]` idiom used throughout `src/` ensures that discovery works from any working directory.

Graceful fallbacks over exceptions is the correct trade-off for a resource-consumption layer. The alternative — raising `FileNotFoundError` when a fond is missing — would make the integration pipeline fragile to the ordering of parallel agent writes. Returning `None` and logging a warning allows the pipeline to produce a complete, if partial, result dict that downstream consumers can reason about.

Design Decisions Revisited I

Real-file tests over mocks is required for a template exemplar. Mocks can pass tests while the real discovery logic is silently broken. Tests that use real file paths exercise the full path-resolution chain and catch regressions that mocks would miss.

Thin orchestration scripts keep the scripts directory readable and focus all testable logic in `src/`. A script that does more than parse arguments, call a `src/` function, and write output is accumulating business logic that belongs elsewhere.

Limitations I

This exemplar makes several deliberate simplifications that a reader adopting the pattern should be aware of:

Limitations I

- **Structural, not semantic, validation by default.** `validate_against_rules()` and `discover_tools()` confirm that manifests and rule files are parseable YAML with the expected top-level shape; they do not check that a fond's *data* conforms to its declared schema, or that a tool's entrypoint script actually behaves according to its documented contract. Semantic constraint checking exists only for strong rules, via the separate `strong_rule_evaluator` module (sec. 5), and only for the constraint kinds that module implements: `coverage_threshold`, `module_structure`, `section_schema`, `reference_schema` (all four wired to a real strong-rule YAML file and exercised against this project's own live tree), plus `manifest_freshness` (evaluator logic implemented and unit-tested, deliberately not yet wired to a real rule file — see Future Directions).
- **No concurrency handling.** All readers assume a single-process, single-read invocation. If a parallel agent is actively writing a fond's `data/` file while another process reads it, the reader may observe a partially-written file and report a spurious parse error rather than a clean "not yet available" signal. The architecture tolerates *absence* gracefully; it does not guarantee *atomicity* against concurrent writers.

Limitations II

- **Small-scale assumption.** As discussed in sec. 7's Performance and Overhead subsection, the design is tuned for a curated, human-reviewed set of resources (single-digit to low-double-digit counts per category), not a resource catalogue at data-lake scale.
- **English-language, code-review-oriented soft rules.** Soft rules are Markdown prose intended for human reviewers and AI coding agents; they are not evaluated by any automated tool in this exemplar, unlike strong rules. A project that wants soft-rule compliance checked automatically would need to promote the relevant guideline to a strong rule with a corresponding evaluator function.
- **`strong_rule_evaluator.py` had the thinnest test coverage of any `src/` module, historically.** As measured in an earlier coverage run, it required more negative-control tests (malformed rule files, real violations, type-mismatched context values) than the other source modules, concentrated in its `section_schema`, `reference_schema`, and `module_structure` evaluators and its context loader. That gap has since been closed with real negative-control fixtures — run `uv run pytest … --cov-report=term-missing` for the current per-module breakdown, since coverage changes as the suite grows.

The no-concurrency-handling limitation above is not merely theoretical: this exemplar is itself designed to be populated by parallel agents authoring `fonds/`, `rules/`, and `tools/` concurrently, which is precisely the scenario where a torn read is most likely. Adopters running this pattern in a genuinely concurrent write environment should add file locking or an atomic-rename write pattern at the resource-authoring layer — this exemplar deliberately does not, to keep the reader-side code minimal.

Future Directions I

The three-layer architecture described here is deliberately extensible. The most natural extension is a fourth resource category: **models** (`models/<scope>/<name>/`) for pre-trained machine learning models and their inference scripts. The pattern would follow exactly the same manifest-and-reader design, with `models.yaml` declaring type, version, and entrypoints, and a `models_loader` module providing `discover_models()` and `validate_model_files_exist()`. Adding this layer to `template_pools_rules_tools` would require only a new reader module and a new section in the integration orchestrator — the existing architecture places no constraints on the number of resource categories.

A second direction, **cross-fond validation**, is now implemented: `src/template_pools_rules_tools/tools/integration.py::check_bibliography_overlap()` compares this manuscript's own cite keys against the `template_bibliography` fond's curated set and reports `overlap`, `project_only`, and `fond_only` counts. It deliberately does *not* assert full containment in either direction — this manuscript legitimately cites software-engineering sources the fond does not curate, and the fond legitimately curates machine-learning sources this manuscript does not cite. Run `uv run python -c "from template_pools_rules_tools.tools.integration import check_bibliography_overlap; import json; print(json.dumps(check_bibliography_overlap(), indent=2))"` from the project root to see the current overlap.

A third direction, suggested directly by the Limitations above, was extending `strong_rule_evaluator` with additional rule kinds beyond the original four. The `manifest_freshness` kind — flagging a fond whose `version` field has not been bumped despite its `data/` files changing more recently than its manifest — is now implemented: `src/template_pools_rules_tools/rules/strong_rule_evaluator.py::_evaluate_manifest_freshness()` is a fifth dispatch entry, unit-tested against synthetic rule dicts (`tests/rules/test_strong_rule_evaluator.py`). It is deliberately **not** yet wired to a real strong-rule YAML file: doing so would mean adding a file under the shared `rules/templates/template_project_rules/strong/` or `rules/templates/template_manuscript_rules/strong/` directory, which this project's own read-only contract against `fonds/`,

`rules/`, and `tools/` reserves for an explicit decision rather than a routine content edit. The evaluator's logic is proven correct against synthetic input; proving it against a real fond's real manifest and data-file timestamps is the natural next step, pending that decision.

The exemplar is ready to fork. A project team wishing to adopt this architecture need only copy the relevant `src/template_pools_rules_tools/` modules (the three readers plus `integration.py` are the minimum; `strong_rule_evaluator.py` and `figures.py` are optional extensions), update the resource directory paths in each reader, and replace the exemplar fond/rules/tool names with their own.